

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

<https://github.com/VenkatChowdary999/DeepLearning-Labs/tree/main/LAB%20-3>

Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

1. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
 - Activation Layer
 - Pooling Layer
 - Fully Connected Layer
- The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size P
 Padding S
 Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied elementwise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

```

  ?      ?
    0    0
  ?      ?
  ? 0    0 ? ?
  ? 0    1 ?
    0 0 0
  
```

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

```

  ?      ?
    -1    1
  ?      ?
  ? -2    0 2 ? ?
  ?      0 ?
    -1    0 1
  
```

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

```

  ?      -2      ?
    -1    0    -1
  ? ?      2    0
  0      ? ?
  ?      ? ?
    1      1
  
```

Applications:

- Text detection

- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ 0 & 4 & 0 \\ 0 & -1 & -1 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ 0 & 5 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -2 & 1 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 8 & -1 \\ 0 & -1 & -1 \\ 0 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$\begin{array}{ccc}
 \boxed{?} & & \boxed{?} \\
 1 & 2 & 3 \\
 \\
 X = \boxed{?}\boxed{?}4 & 5 & 6\boxed{?}\boxed{?} \\
 \\
 \boxed{?} & & \boxed{?} \\
 7 & 8 & 9
 \end{array}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{array}{cccc} 1 & 5 & 2 & 3 \\ & & & \\ & & & \\ & & & \end{array} \begin{array}{c} \\ \\ \\ \end{array} \begin{bmatrix} 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \end{bmatrix}$$

Using a 2×2 max pooling filter, Window 1

$$\begin{array}{cc} " & \# \\ 1 & 5 \\ & \rightarrow 8 \\ 7 & 8 \end{array}$$

Window 2

$$\begin{array}{cc} " & \# \\ 2 & 3 \\ & \rightarrow 3 \\ 1 & 0 \end{array}$$

Window 3

$$\begin{array}{cc} " & \# \\ 4 & 6 \\ & \rightarrow 6 \\ 2 & 3 \end{array}$$

Window 4

$$\begin{array}{cc} " & \# \\ 9 & 5 \\ & \rightarrow 9 \\ 1 & 8 \end{array}$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$\begin{aligned} &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

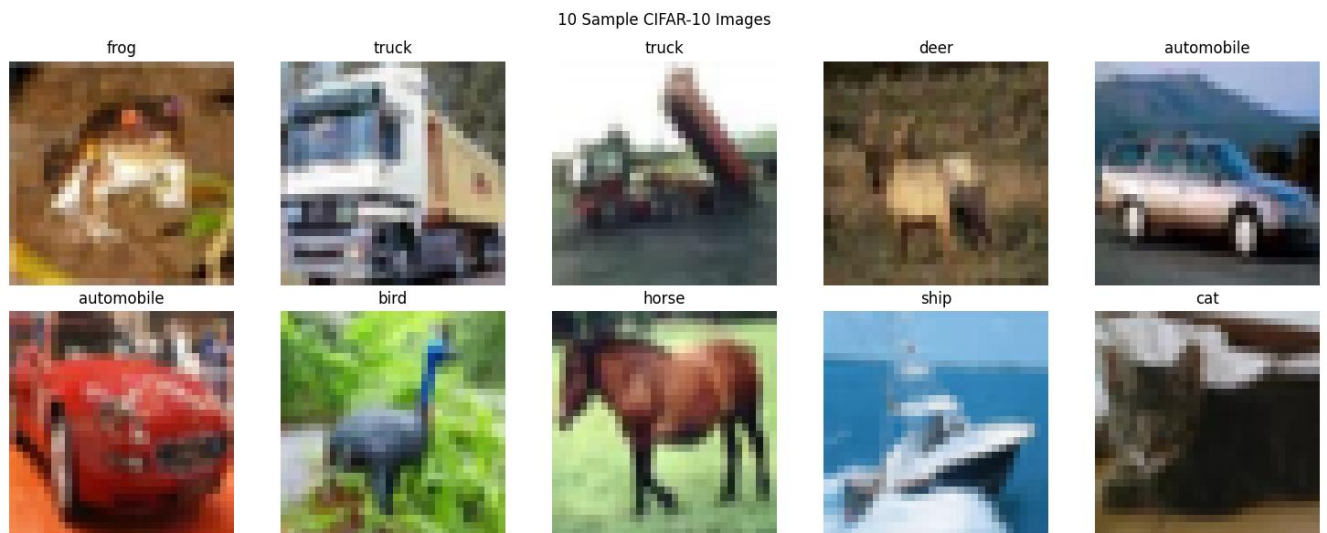
- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.

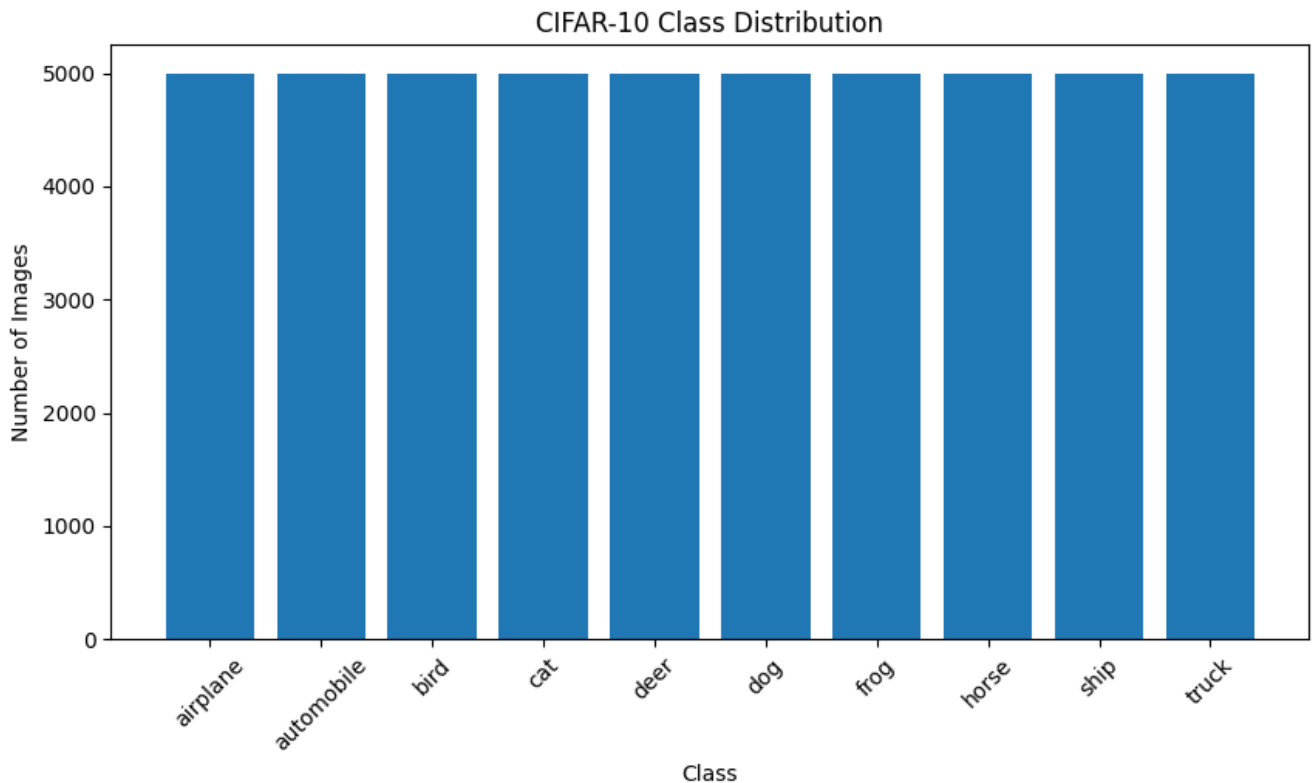


Print dataset dimensions.

```
print("Training images :", X_train.shape[0])
print("Testing images  :", X_test.shape[0])
print("Image size      :", X_train.shape[1:])
print("Number of classes:", len(class_names))
```

```
Training images : 50000
Testing images  : 10000
Image size      : (32, 32, 3)
Number of classes: 10
```

- Plot class distribution.



The class distribution is approximately balanced, with each of the ten classes containing nearly the same number of images. Therefore, there is no significant class imbalance in the CIFAR-10 dataset.

Task 2

Implement a convolution layer. Compare

- Kernel = 3×3 • Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

```
3x3: (1, 30, 30, 16)
5x5: (1, 28, 28, 16)
7x7: (1, 26, 26, 16)
```

As the kernel size increases from **3×3 to 7×7** , the spatial dimensions of the feature map decrease when valid padding is used. Larger kernels capture features over a wider receptive field but require more computational resources and trainable parameters. The **3×3 kernel** provides a good balance between feature extraction and computational efficiency.

Task 3

Study hyperparameters. Compare

- Stride = 1
- Stride = 2
- Padding = Same • Padding = Valid

Compute output dimensions.

3x3 kernel

Stride 1, Valid: 30 x 30

Stride 2, Valid: 15 x 15

Stride 1, Same: 32 x 32

Stride 2, Same: 16 x 16

Valid, stride 1: (1, 30, 30, 16)

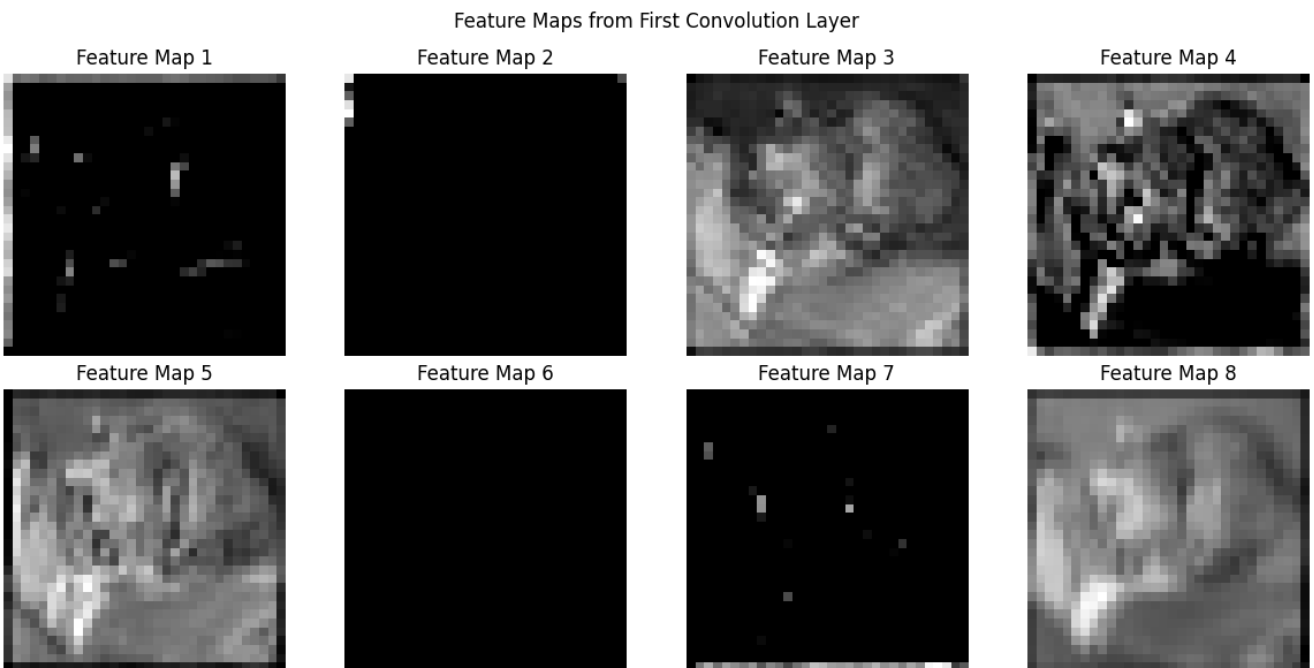
Same, stride 1 : (1, 32, 32, 16)

Same, stride 2 : (1, 16, 16, 16)

Increasing the **stride** reduces the spatial dimensions of the feature map because the filter moves by larger steps. **Same padding** preserves the spatial dimensions when stride is 1, whereas **valid padding** reduces the dimensions because no padding is added.

Task 4

Visualize feature maps after the first convolution layer. Display at least eight feature maps.



The feature maps generated by the first convolutional layer show the visual features detected by different filters. These filters learn low-level characteristics such as **edges, textures, shapes, and intensity patterns** from the input image.

Task 5 Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

```
Original : (1, 32, 32, 3)
Max Pool : (1, 16, 16, 3)
Avg Pool : (1, 16, 16, 3)
```

1. Output Dimension

Both Max Pooling and Average Pooling reduce the spatial dimensions of the feature map by a factor determined by the pooling size. For a **2×2 pooling window**, a 32×32 feature map is reduced to 16×16.

2. Visual Comparison

Max Pooling selects the maximum activation from each pooling region and therefore preserves the strongest detected features. Average Pooling calculates the average value and produces a smoother representation of the feature map.

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 32, 32, 16)	448
re_lu (ReLU)	(None, 32, 32, 16)	0
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 16)	0
conv2d_8 (Conv2D)	(None, 16, 16, 32)	4,640
re_lu_1 (ReLU)	(None, 16, 16, 32)	0
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dense_1 (Dense)	(None, 10)	1,290

Total params: 268,650 (1.02 MB)
Trainable params: 268,650 (1.02 MB)
Non-trainable params: 0 (0.00 B)

- The CNN consists of convolutional layers for feature extraction, ReLU activation for introducing non-linearity, max-pooling layers for dimensionality reduction, a flattening layer for converting feature maps into a vector, and dense layers for classification. The final Softmax layer produces probabilities for the **10 CIFAR-10 classes**.

Task 7

Evaluate

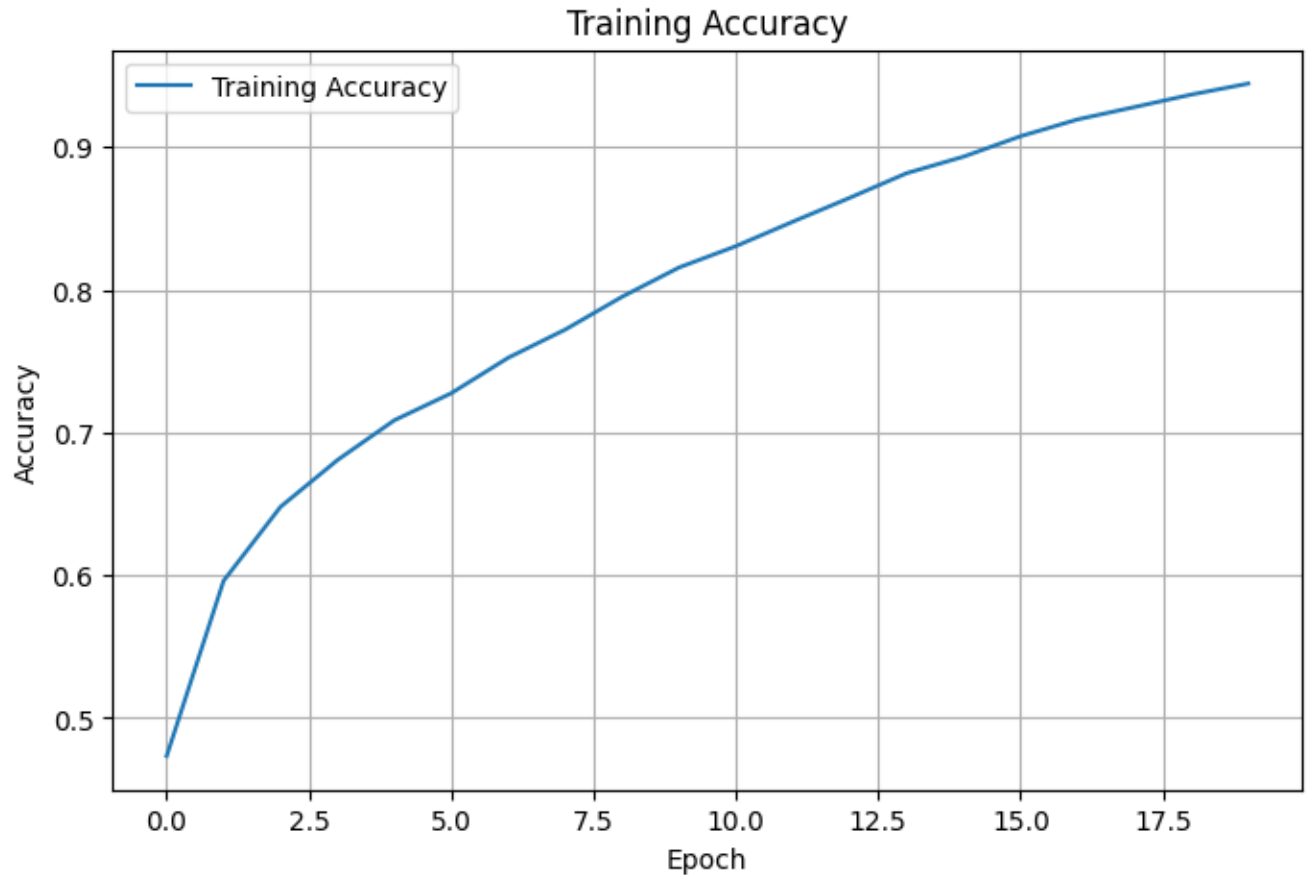
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

```
Accuracy : 0.662
Precision: 0.6769298746510575
Recall   : 0.662
F1-score : 0.6661746588713855
```

9. Mandatory Plots

Generate

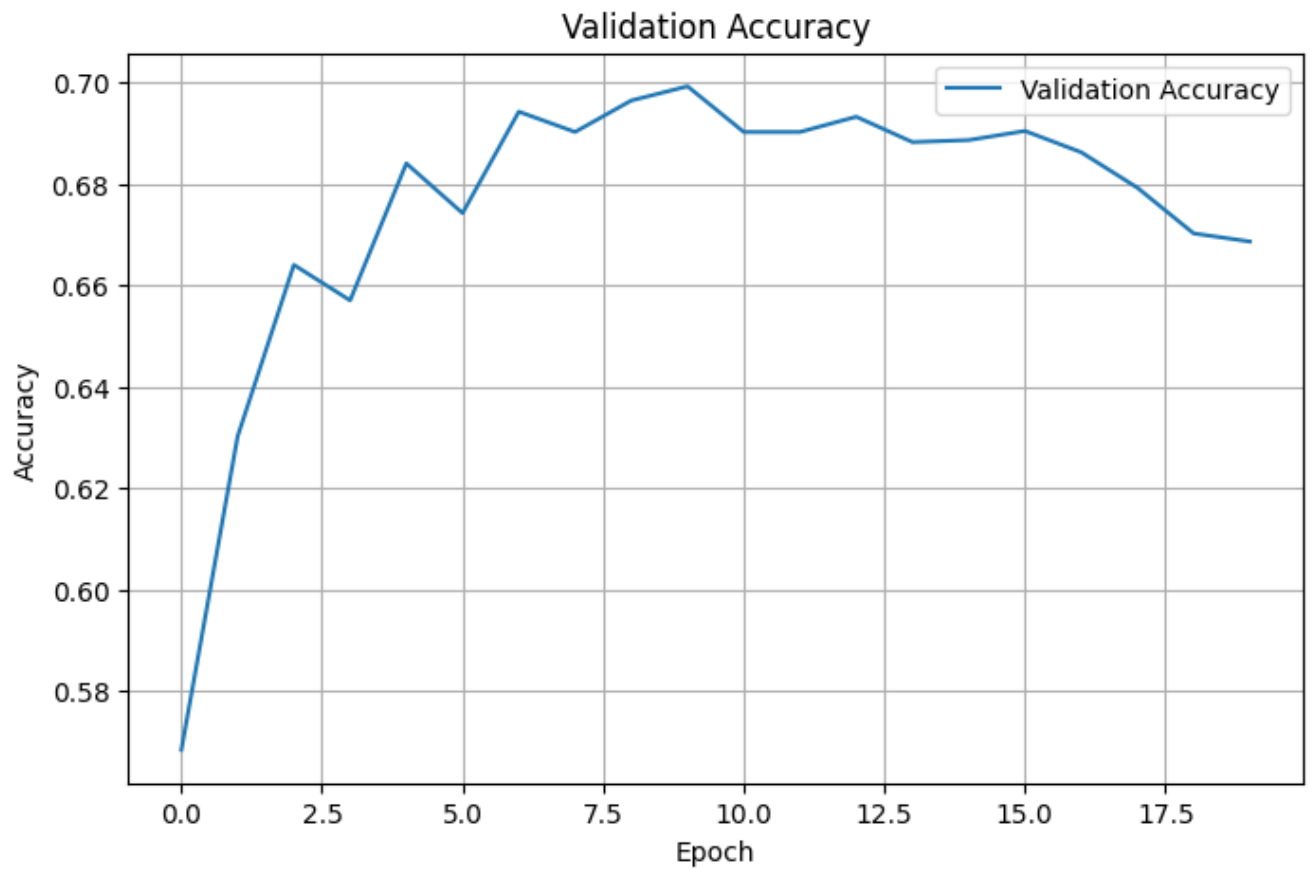
1. Sample Images
2. Class Distribution
3. Training Accuracy



4.

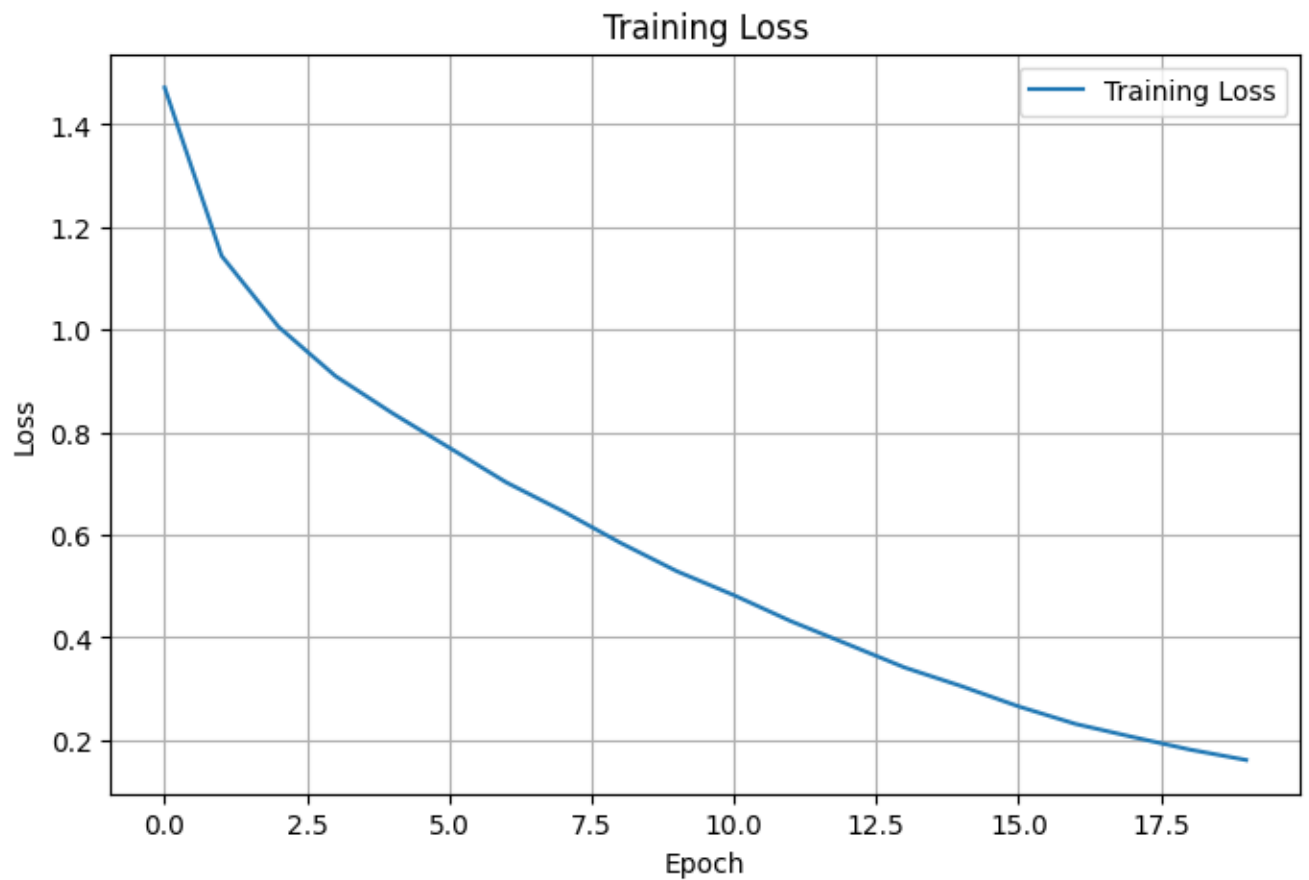
10. The training accuracy generally increases as the number of epochs increases, indicating that the CNN is learning useful features from the training images. A high training accuracy indicates effective learning from the training dataset.

1. Validation Accuracy



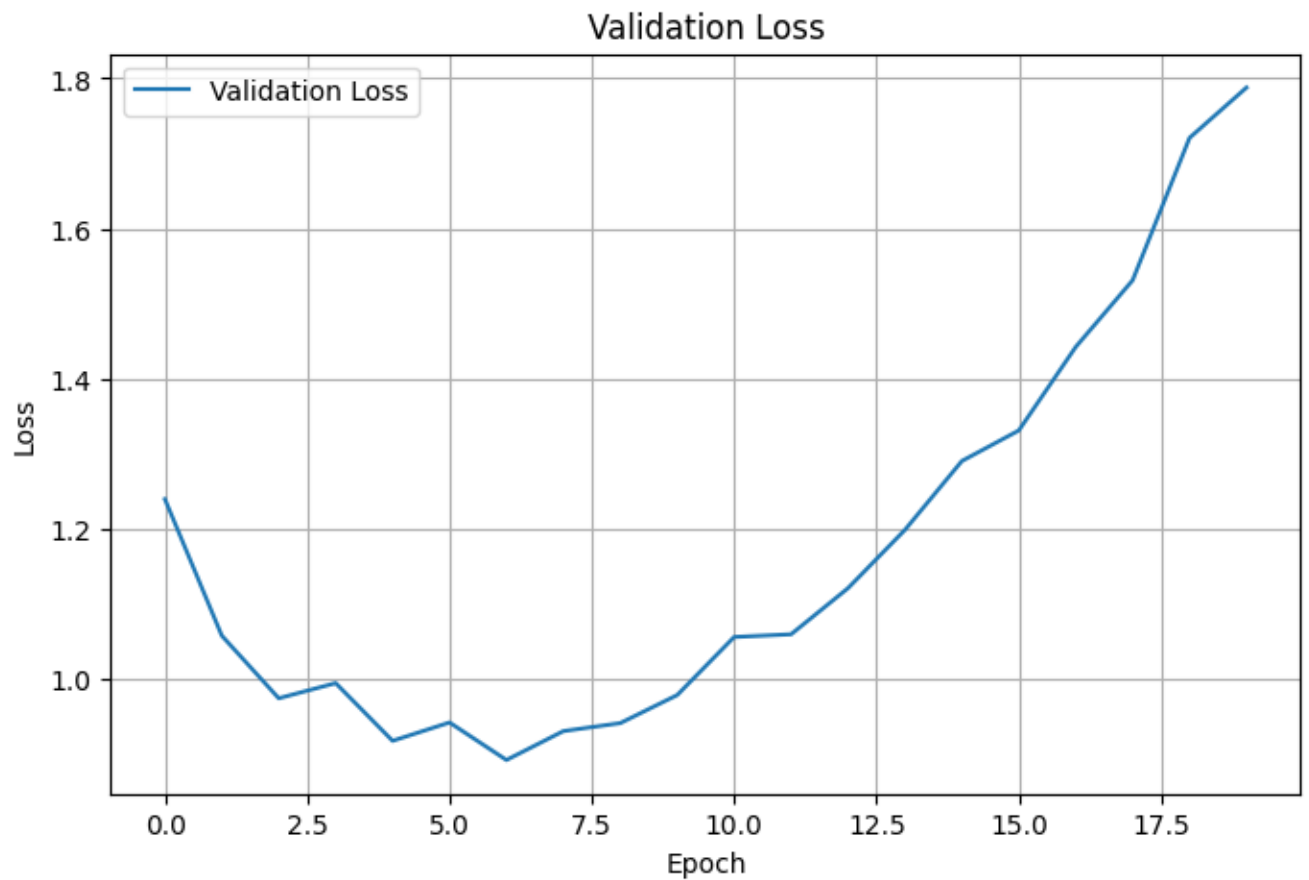
The validation accuracy indicates how well the model generalizes to unseen images. An increasing and relatively stable validation accuracy suggests that the CNN is learning features that generalize beyond the training data.

2. Training Loss

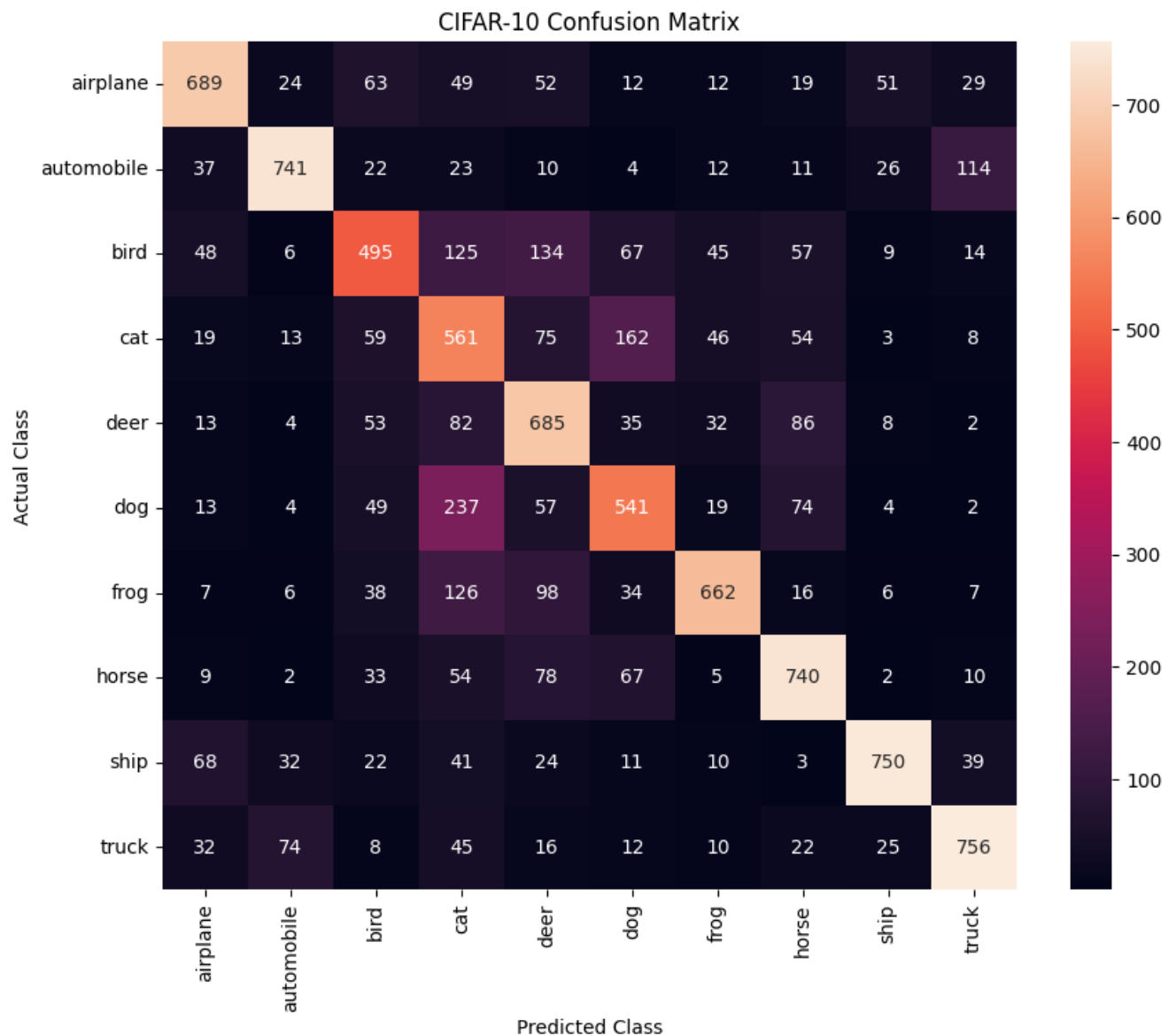


The training loss generally decreases as training progresses, indicating that the difference between the predicted and actual class labels is reducing. This demonstrates that the optimizer is effectively updating the network weights.

3. Validation Loss



- 4.
5. The validation loss indicates the model's performance on unseen validation data. A decreasing validation loss suggests good generalization, while an increasing validation loss together with decreasing training loss may indicate overfitting.
6. Feature Maps
7. Confusion Matrix



- 8.
9. The confusion matrix shows the relationship between actual and predicted classes. The diagonal elements represent correctly classified images, while the off-diagonal elements represent misclassifications between different object categories.

Write a 2-3 line inference for each plot.

11. Additional Exercises

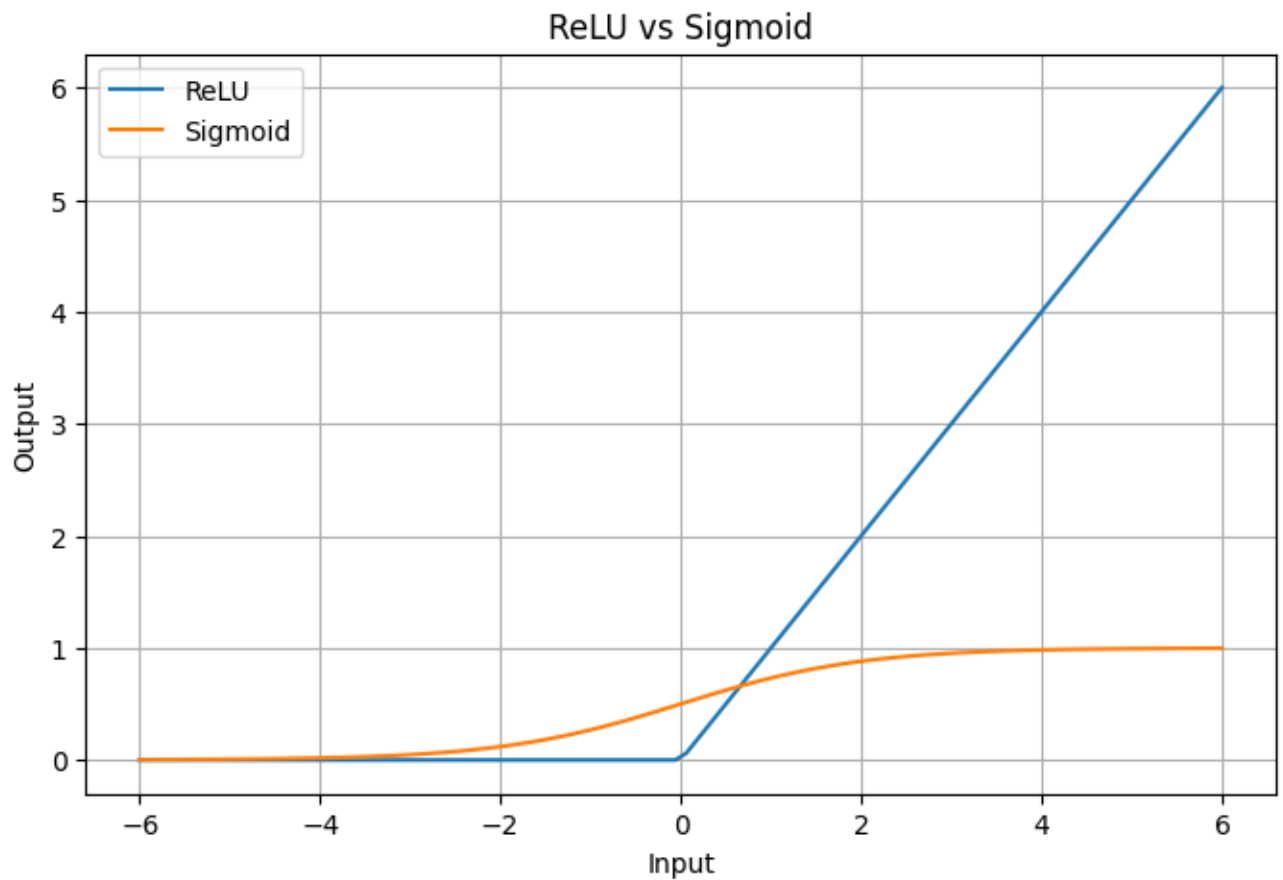
1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Total model parameters: 268650

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

Total model parameters: 268650

3. Compare ReLU and Sigmoid activation functions.



- Replace Max Pooling with Average Pooling and compare the results.

```
Max Pooling Accuracy : 0.6620000004768372
Average Pooling Accuracy: 0.6819999814033508
```

- Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

```
FILTER COMPARISON
=====
16-filter model
Accuracy : 0.6620000004768372
Parameters : 268650

64-filter model
Accuracy : 0.6875
Parameters : 564426
Training Time: 128.50361514091492 seconds
```

```
64-filter Test Accuracy: 0.6875
64-filter Parameters: 564426
```

12. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score

Number of Parameters

FINAL RESULTS	
Training Accuracy	: 0.9444
Testing Accuracy	: 0.6620
Precision	: 0.6769
Recall	: 0.6620
F1-score	: 0.6662
Parameters	: 268650

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?
2. How does stride affect the feature map size?
3. What is the role of padding?
4. Why is pooling used?
5. How do feature maps represent image characteristics?
6. Why do CNNs require fewer parameters than MLPs?

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.